


# Practical Report – Group No. 4

# IE3064 Embedded Systems Design

# 1st year 3rd semester - 2026

Group Number: 04

Student IDs: IT24103478 – M. V. M Fernando

IT24101650 – M. T. M Dushan

# Contents

1. Introduction
2. Component Selection and Justification
3. Hardware Design
4. Firmware Architecture
5. State Machine Diagram
6. Subroutine Flowcharts
7. Phase Duration Table
8. Implementation Details
9. Testing and Results
10. Photographs
11. Discussion
12. Individual Contribution Statements
References

# 1. Introduction

This project is a prototype of a four-way road intersection that is being operated by Arduino Uno (ATmega328P). The software has been programmed using AVR Assembly language. There is no usage of any C, C++, or Arduino library function in the code.

The project includes four roads (North, South, East, and West) with red, yellow, and green LEDs. The push button on the South road requests a pedestrian crossing while the LDR sensor indicates night time..

**Objectives**

• Implement the four-phase cycle in Normal mode that has a yellow phase and an all-red phase between two directions of traffic.

• Delay the pedestrian button until the current phase of vehicles completes itself along with the all-red period, then make all phases red for a 5 second countdown from 5 to 0.

• Switch to Night mode if the Light Dependent Resistor reading drops below a certain level, light up all yellow LEDs at 1 Hz frequency, and switch back to Normal mode after 5 seconds of continuous lighting (hysteresis).

• Implement the use of multiple I/O ports, sequencer based on timers, interrupts-based events, analog sensing and countdown timer using AVR Assembly.**Toolchain.** The firmware is a single GNU assembler source file, traffic.S, built with avr-gcc -mmcu=atmega328p -nostdlib -nostartfiles and uploaded with avrdude. The header avr/io.h supplies register names only; no C code is compiled.

# 2. Component Selection and Justification

The model uses one Arduino Uno, 17 LEDs with 330 Ω resistors, one push-button and one LDR with a 10 kΩ divider resistor.

| Component | Qty | Justification |
|---|---|---|
| Arduino Uno (ATmega328P) | 1 | Required by the assignment. Provides GPIO, external interrupt INT0, a 10-bit ADC and a 16-bit timer. |
| LED, red | 4 | Traffic lights, one per approach. |
| LED, yellow | 4 | Traffic lights, one per approach. Also used for Night mode flashing. |
| LED, green | 4 | Traffic lights, one per approach. |
| LED, countdown | 5 | Pedestrian countdown, turned off one by one from 5 to 0. |
| Resistor 330 Ω | 17 | One per LED. Limits LED current to about 9 mA. |
| Push-button | 1 | Pedestrian request on the South approach. Uses the internal pull-up, so no external resistor is needed. |
| LDR | 1 | Ambient light sensing for Night mode. |
| Resistor 10 kΩ | 1 | Lower half of the LDR voltage divider. |
| Power supply | 1 | Battery Pack |
| Base board | 1 | 60 × 60 cm cardboard |

**LED resistor.** With a 5 V supply, an LED forward voltage of about 2 V and a target current of about 9 mA:

R = (Vs − Vf) / If = (5 − 2) / 0.009 ≈ 333 Ω, so 330 Ω is used, giving I = 3 / 330 ≈ 9.1 mA.

**Pin current check:** Since there are two LEDs wired in parallel for each traffic pin, each pin uses 18 mA of current, which is below the 20 mA recommendation. During the pedestrian phase, two red pins (36 mA) and five countdown LEDs (45 mA) are working, totaling to 81 mA, which is below 200 mA capacity of the chip**.**

**LDR divider.** The LDR is connected from 5 V to A0, and a 10 kΩ resistor from A0 to GND:

V(A0) = 5 × 10 kΩ / (R_LDR + 10 kΩ)

In the dark R_LDR rises, so the voltage at A0 and the ADC value fall. The 8-bit ADC value is therefore small in the dark and large in the light.

**Shared pins.** The design needs 19 I/O lines (12 traffic LEDs, 5 countdown LEDs, the button and the LDR), but the Uno has 16 usable pins once PD0 and PD1 are left free for serial upload. North and South always show the same colour, as do East and West, so each pair shares three pins. All 12 traffic LEDs are still physically present.

# 3. Hardware Design

Each traffic light, each countdown light, the push button, and the LDR have been connected to fixed Arduino pins shown in Table 1 with AVR port names. Block diagram is shown in Figure 1, circuit constructed in Tinkercad is illustrated in Figure 2, and Figure 3 and 4 show the schematic diagram exported from Tinkercad.

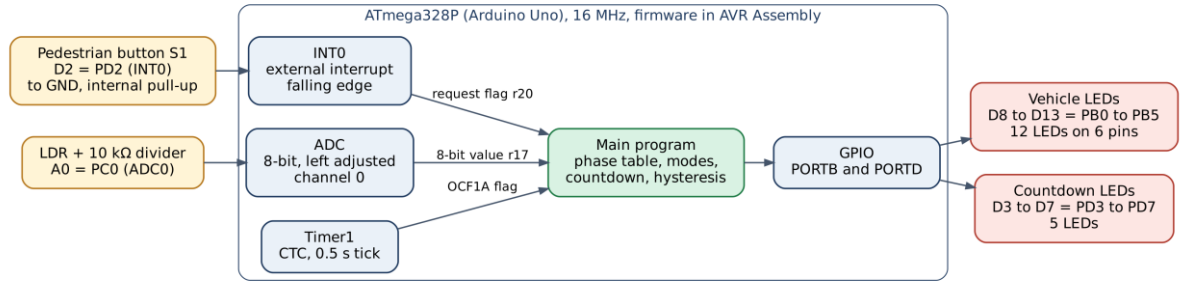



*Figure 1: Block diagram of the system*

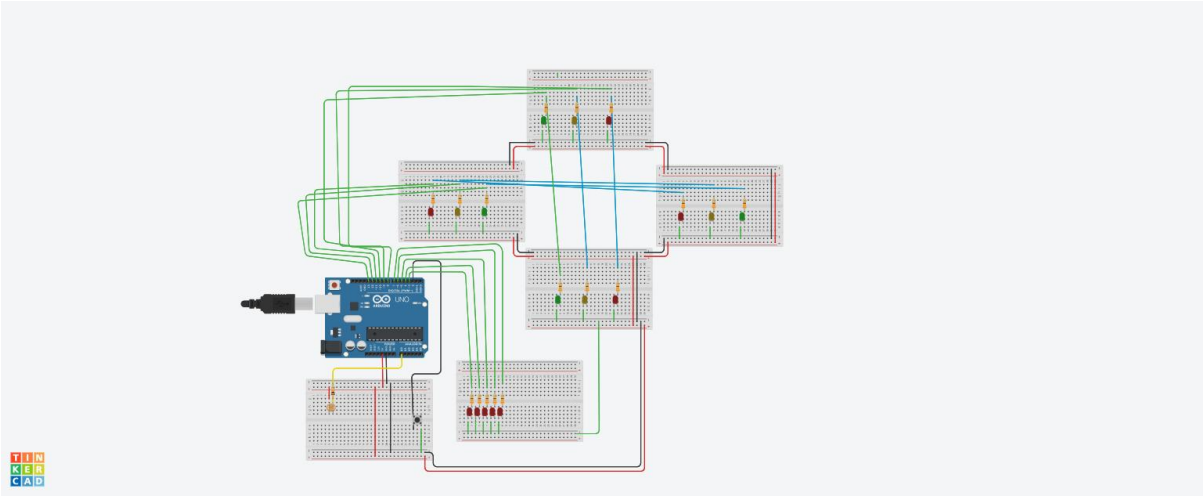


*Figure 2: Circuit built in Tinkercad (breadboard view)*

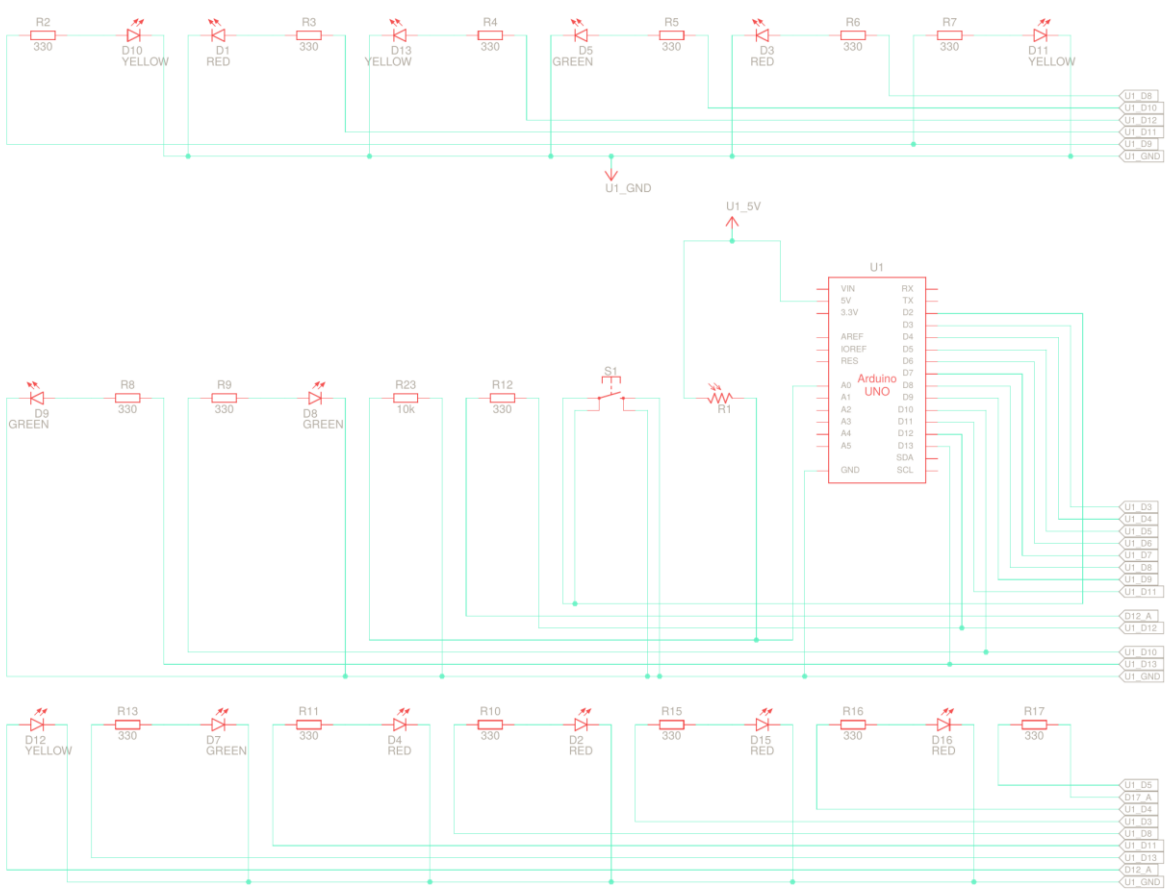



*Figure 3: Schematic, sheet 1 of 2 (traffic LEDs, button, LDR and Arduino Uno), exported from Tinkercad*

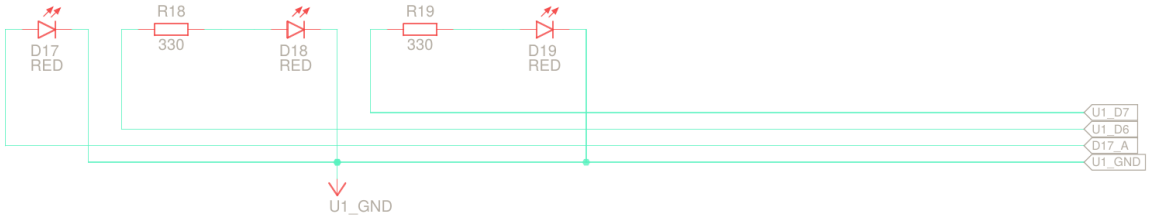



*Figure 4: Schematic, sheet 2 of 2 (remaining countdown LEDs D17 to D19), exported from Tinkercad*

**Table 1: Pin mapping**

| Arduino pin | AVR port | Direction | Function |
|---|---|---|---|
| D8 | PB0 | Output | North + South RED |
| D9 | PB1 | Output | North + South YELLOW |
| D10 | PB2 | Output | North + South GREEN |
| D11 | PB3 | Output | East + West RED |
| D12 | PB4 | Output | East + West YELLOW |
| D13 | PB5 | Output | East + West GREEN |
| D2 | PD2 (INT0) | Input, pull-up | Pedestrian push-button to GND |

| Arduino pin | AVR port | Direction | Function |
|---|---|---|---|
| D3 | PD3 | Output | Countdown LED 1 |
| D4 | PD4 | Output | Countdown LED 2 |
| D5 | PD5 | Output | Countdown LED 3 |
| D6 | PD6 | Output | Countdown LED 4 |
| D7 | PD7 | Output | Countdown LED 5 |
| A0 | PC0 (ADC0) | Analogue input | LDR voltage divider midpoint |

**Wiring notes**

• Each LED has its own 330 Ω resistor connected to it in series. The two LEDs from the same pair (North and South, or East and West) are connected in parallel to the same Arduino pin.

• The push-button has D2 connected to GND. There is an internal pull-up that keeps D2 HIGH until the button is pressed. This results in a falling edge for INT0.

• LDR has 5V connected to A0, while a 10 kΩ resistor is connected between A0 and GND.

• The pedestrian button and the countdown LEDs are mounted on the South approach. The LDR is located in such a way that it can be covered by a hand.

# 4. Firmware Architecture

The firmware is one assembly file. A main program steps through a phase table, and one interrupt service routine handles the pedestrian button.

**Table 2: Program structure**

| Part | Labels | Role |
|---|---|---|
| Vector table | RESET, BUTTON_ISR | Reset vector at 0x0000, INT0 vector at 0x0004. |
| Initialisation | RESET | Stack pointer, ports, INT0, ADC, Timer1, then sei. |
| Normal mode | NORMAL_MODE, PHASE_LOOP | Reads the phase table with lpm and drives PORTB. |
| Timing | NORMAL_DELAY, WAIT_ONE_SECOND, HALF_SECOND | Waits on the Timer1 compare flag and samples the LDR every 0.5 s. |
| ADC | READ_ADC | One conversion on ADC0, 8-bit result. |
| Pedestrian mode | PEDESTRIAN_MODE | All-red crossing phase and 5 to 0 countdown. |
| Night mode | ENTER_NIGHT, NIGHT_LOOP, NIGHT_HALF, NH_EXIT | 1 Hz flashing and 5 s exit hysteresis. |
| Interrupt | BUTTON_ISR | Sets the pedestrian request flag only. |
| Data | PHASE_TABLE | Phase patterns and durations in program memory. |

**Table 3: Register usage**

| Register | Purpose |
|---|---|
| r16, r23, r25 | Temporary values and loop counters |
| r17 | Latest 8-bit ADC result |
| r18 | Seconds remaining in the current phase |
| r19 | Current phase index (0 to 5) |
| r20 | Pedestrian request flag (0 or 1) |
| r22 | Dark flag, set when the ADC value is below DARK_TH |
| r24 | Consecutive bright half-seconds in Night mode |
| Z (r31:r30) | Pointer into the phase table for lpm |

**Table 4: Constants (top of traffic.S)**

| Constant | Value | Meaning |
|---|---|---|
| DARK_TH | 50 | ADC value below this means dark (about 0.98 V at A0). Calibrated, see Section 9.1. |
| BRIGHT_TH | 100 | ADC value at or above this counts as bright (about 1.96 V at A0). Calibrated, see Section 9.1. |
| EXIT_COUNT | 10 | 10 samples x 0.5 s = 5 s of light needed to leave Night mode. |

**Design decisions**

- **Table-driven phases.** Light patterns and durations live in program memory, so timings change in one table without touching the logic.
- **Short ISR.** The button interrupt only sets a flag. The main loop acts on it after the all-red interval, which gives the required deferred handling.
- **Dark check every 0.5 s.** HALF_SECOND samples the LDR, so Night mode starts within 0.5 s in any phase.
- **Stack reset on entering Night mode.** Exiting out of the nested calls pushes the return addresses into the stack, which is why ENTER_NIGHT uses RAMEND for the stack pointer.

# 5. State Machine Diagram

Normal mode runs the six-phase cycle. A pedestrian request starts the crossing only after an all-red phase. Darkness sends the system to Night mode from Normal mode or from the crossing, and Night mode returns to Normal mode after 5 s of light followed by 1 s of all-red (Figure 5).

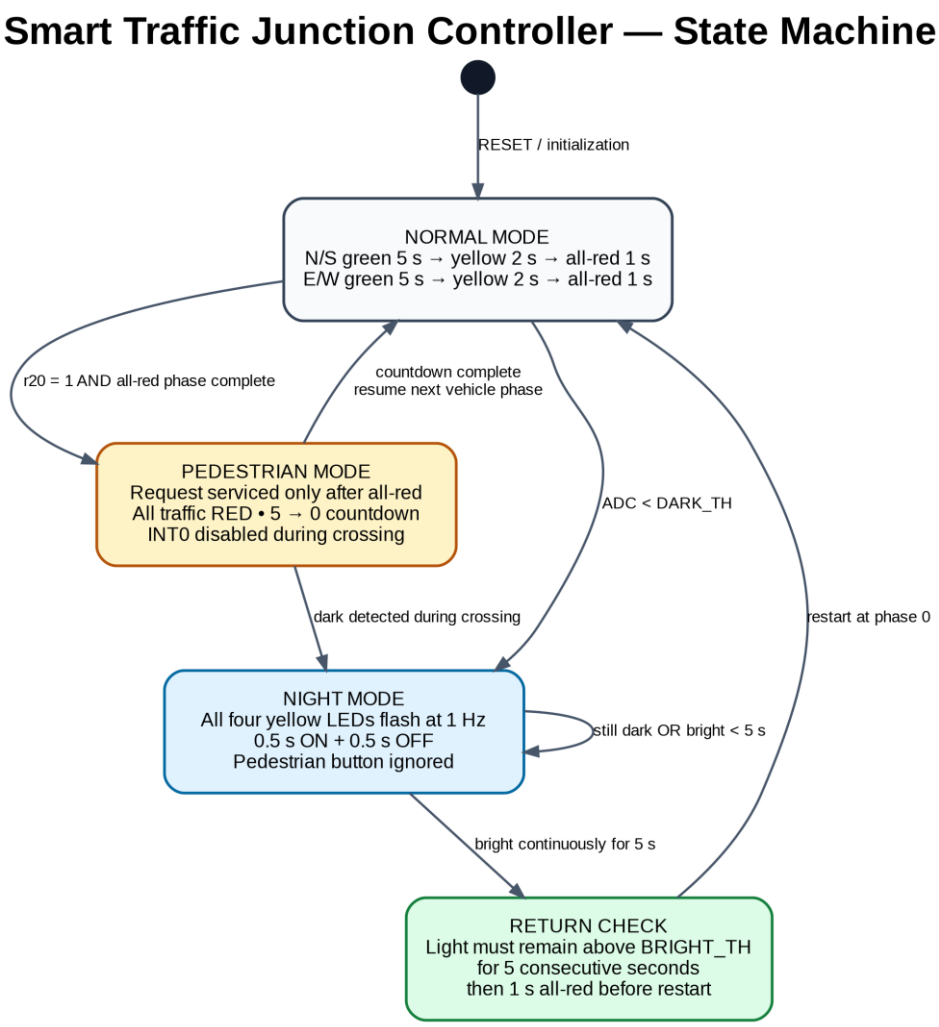



*Figure 5: State machine of the controller*

## 6. Subroutine Flowcharts

Four flowcharts cover the key routines: phase cycling, pedestrian handling, Night mode detection and the countdown display.

## 6.1 Phase cycling

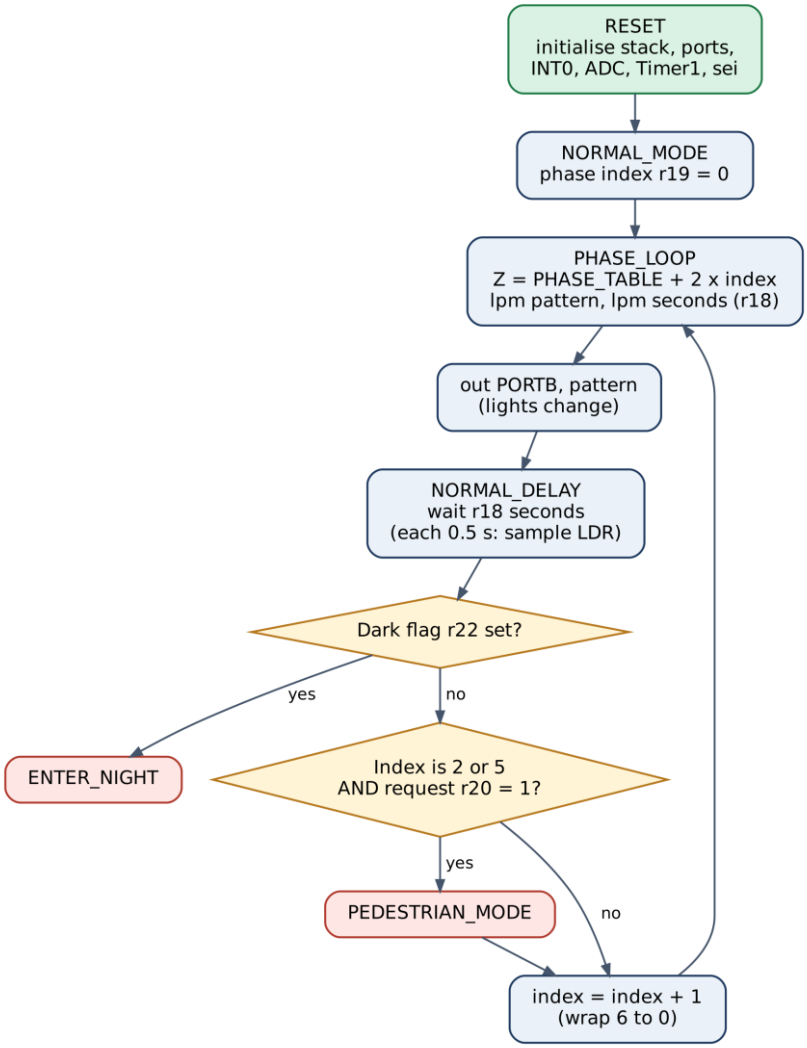



*Figure 6: Flowchart of phase cycling (Normal mode)*

Each pass reads one row of the phase table, drives PORTB, waits, then checks the dark flag and the pedestrian request before advancing.

## 6.2 Pedestrian handling

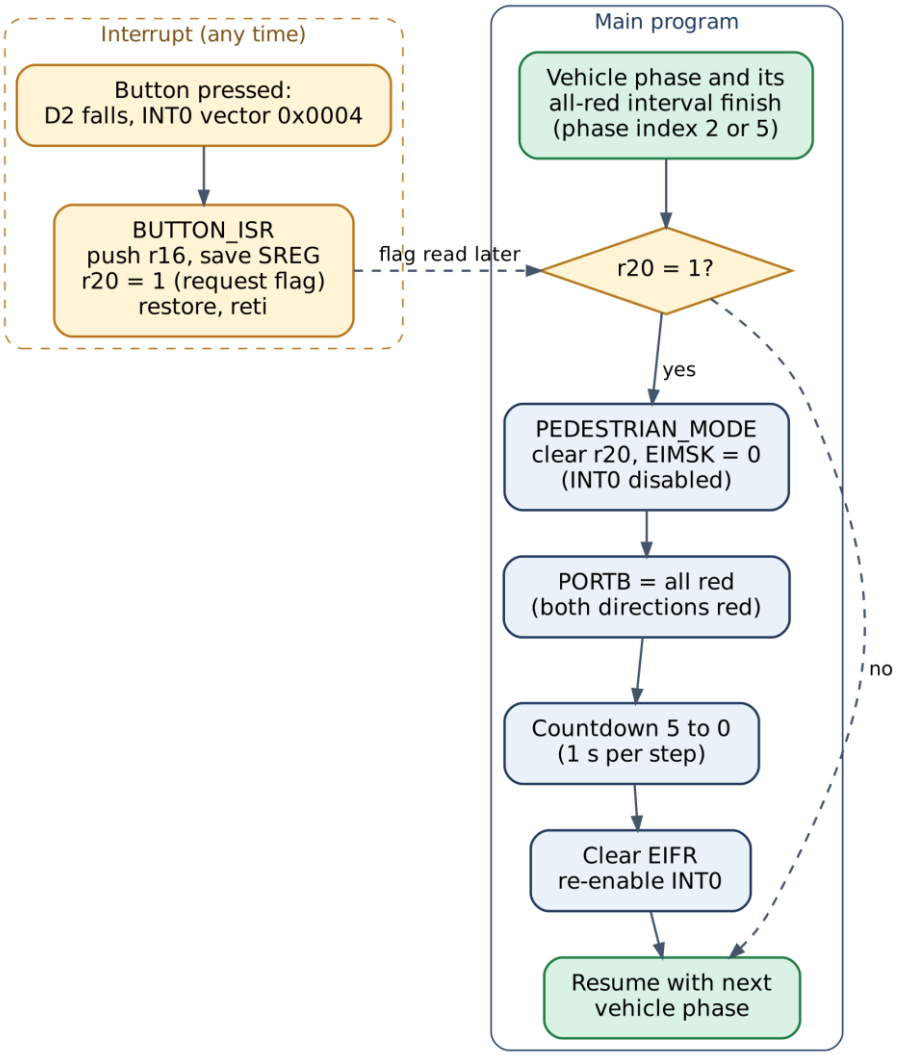



*Figure 7: Flowchart of pedestrian request handling*

The interrupt only records the request. The main loop acts on it after an all-red phase, and clearing r20 and EIFR once means one press gives one crossing.

## 6.3 Night mode detection

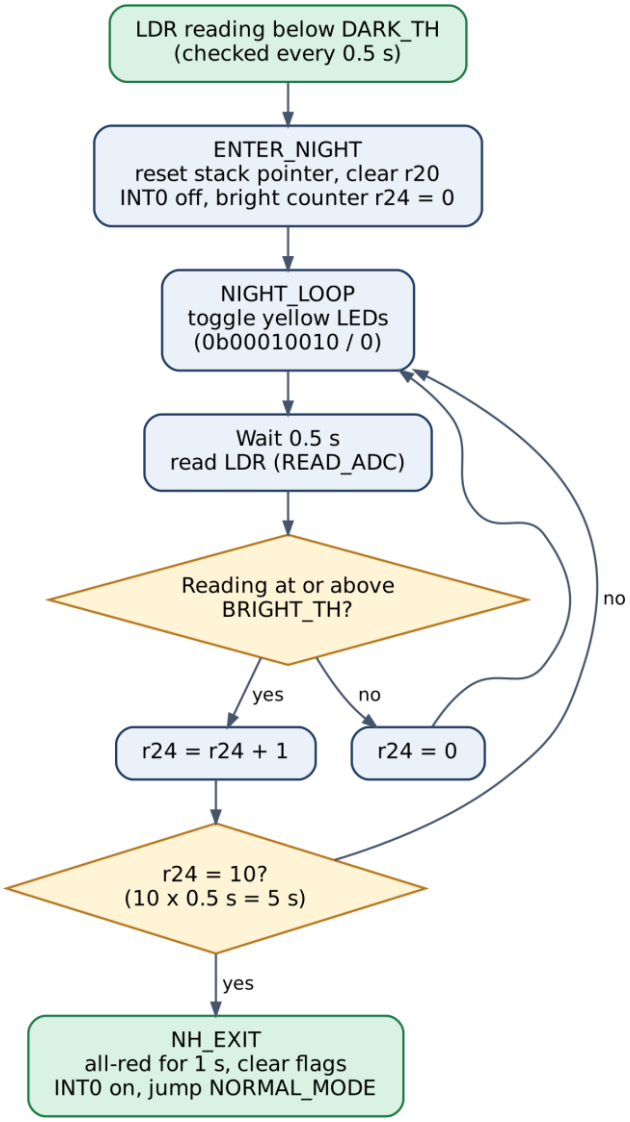



*Figure 8: Flowchart of Night mode and its exit hysteresis*

Every 0.5 s the LDR is read. The light must stay at or above the bright threshold for 10 samples in a row (5 s) before Normal mode returns, and a single lower sample resets the count.

### 6.4 Countdown display

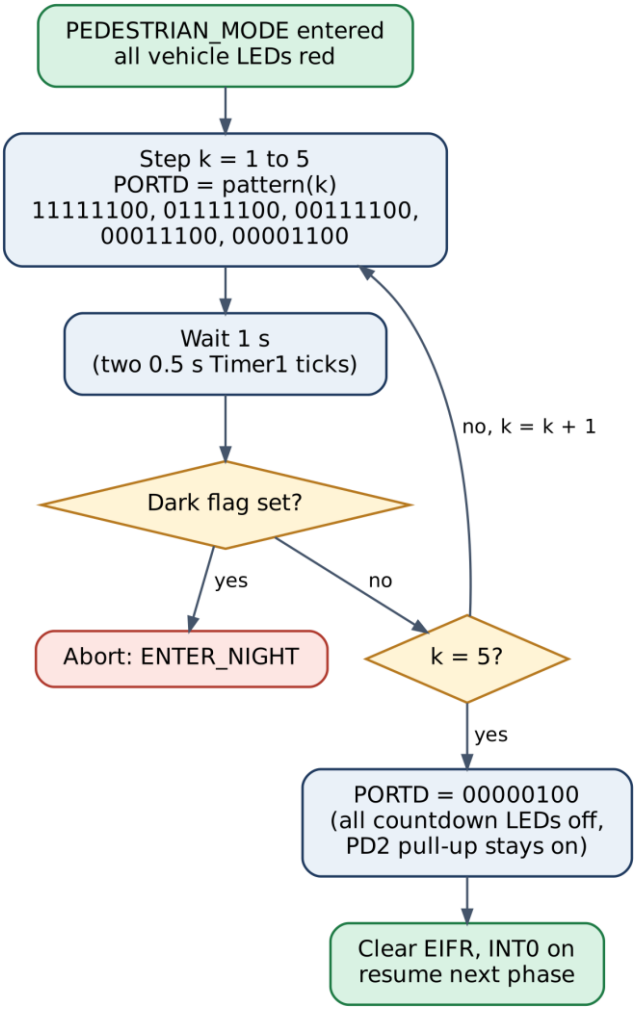



*Figure 9: Flowchart of the pedestrian countdown*

Each step lasts one second, so the five lit steps take 5 s. If the dark flag is set after any step, the countdown stops early and Night mode starts.

## 7. Phase Duration Table

One full cycle takes 16 s: 5 s green, 2 s yellow and 1 s all-red for North-South, then the same for East-West (Figure 10).

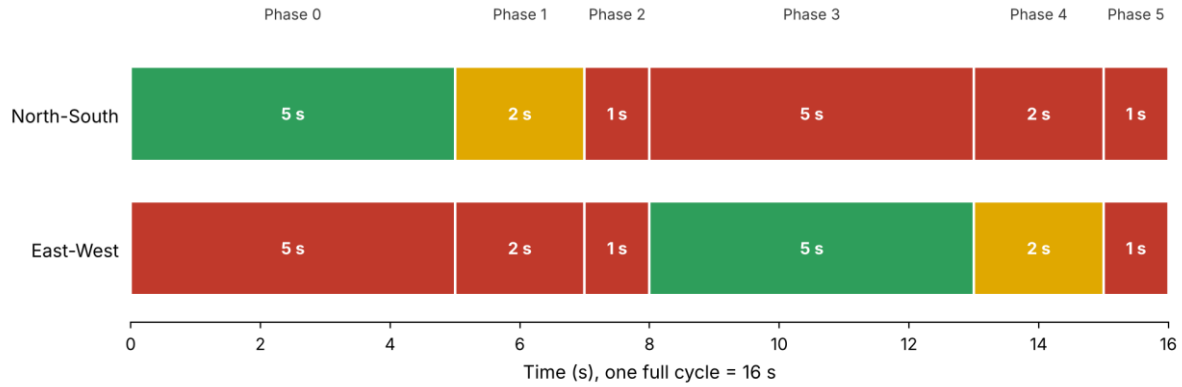



*Figure 10: Timing of one full Normal-mode cycle*

The table is stored in program memory as PHASE_TABLE. Each row is two bytes, the PORTB pattern then the duration in seconds, and the code reads row n with lpm using Z = table address + 2n.

**Table 5: Phase table**

| Phase | North-South | East-West | PORTB pattern | Duration (s) |
|---|---|---|---|---|
| 0 | Green | Red | 0b00001100 | 5 |
| 1 | Yellow | Red | 0b00001010 | 2 |
| 2 | Red | Red (all-red) | 0b00001001 | 1 |
| 3 | Red | Green | 0b00100001 | 5 |
| 4 | Red | Yellow | 0b00010001 | 2 |
| 5 | Red | Red (all-red) | 0b00001001 | 1 |

PORTB bits: bit 0 NS red, bit 1 NS yellow, bit 2 NS green, bit 3 EW red, bit 4 EW yellow, bit 5 EW green.

A pedestrian request is served after phase 2 or phase 5, and the cycle then continues with the next phase. Night mode has no table: PORTB alternates between 0b00010010 (both yellows on) and 0b00000000 every 0.5 s.

# 8. Implementation Details

Four peripherals do the work: Timer1 for timing, INT0 for the button, the ADC for the LDR, and the I/O ports for the LEDs.

## 8.1 Timer1 (0.5 s tick)

Timer1 runs in CTC mode with a prescaler of 256, so it counts 62,500 ticks per second:

16 MHz / 256 = 62,500 Hz;  0.5 s × 62,500 = 31,250 ticks;  OCR1A = 31,250 − 1 = 31,249

| Register | Value | Effect |
|---|---|---|
| TCCR1A | 0 | Normal pin operation |
| TCCR1B | (1<<WGM12) \| (1<<CS12) | CTC mode, prescaler 256, timer starts |

| Register | Value | Effect |
|---|---|---|
| OCR1A | 31249 (0x7A11) | Compare value, written high byte first |

HALF_SECOND clears TCNT1, clears the OCF1A flag, waits until OCF1A is set, clears it again, then reads the LDR. WAIT_ONE_SECOND calls it twice, and NORMAL_DELAY repeats that for the phase duration held in r18.

## 8.2 Pedestrian button (INT0)

```
    ldi  r16, (1<<ISC01)              ; INT0 on the falling edge
    sts  EICRA, r16
    ldi  r16, (1<<INT0)               ; enable INT0
    out  _SFR_IO_ADDR(EIMSK), r16
    sei                               ; global interrupt enable

BUTTON_ISR:
    push r16
    in   r16, _SFR_IO_ADDR(SREG)      ; save status register
    push r16
    ldi  r20, 1                       ; set request flag
    pop  r16
    out  _SFR_IO_ADDR(SREG), r16      ; restore status register
    pop  r16
    reti
```

| Register | Value | Effect |
|---|---|---|
| DDRD bit 2 | 0 | PD2 is an input |
| PORTD bit 2 | 1 | Internal pull-up on, the pin is HIGH until pressed |
| EICRA | (1<<ISC01) | Interrupt on the falling edge (HIGH to LOW) |
| EIFR | (1<<INTF0) | Clears any stale flag |
| EIMSK | (1<<INT0) | Enables INT0 |

The ISR pushes r16 and SREG, sets r20 = 1, restores them and returns with reti. It never touches the lights. During a crossing PEDESTRIAN_MODE writes 0 to EIMSK, so presses are ignored. At the end it clears EIFR and re-enables INT0, so a press made during the crossing is not queued.

## 8.3 ADC (LDR)

```
    ldi  r16, (1<<REFS0)|(1<<ADLAR)    ; AVcc reference, left adjust, ADC0
    sts  ADMUX, r16
    ldi  r16, (1<<ADEN)|(1<<ADPS2)|(1<<ADPS1)|(1<<ADPS0)
    sts  ADCSRA, r16                   ; ADC on, prescaler 128
```

| Register | Value | Effect |
|---|---|---|
| ADMUX | (1<<REFS0) \| (1<<ADLAR) | 5 V reference, left-adjusted result, channel ADC0 |
| ADCSRA | ADEN \| ADPS2 \| ADPS1 \| ADPS0 | ADC on, prescaler 128 (16 MHz / 128 = 125 kHz) |

READ_ADC sets ADSC, waits until it is cleared by hardware, and reads ADCH. When ADLAR bit is set, ADCH contains 8 MSB which results in values between 0 and 255. One conversion takes 13 ADC clock cycles 13 × 8 µs = 104 µs. If the value is less than DARK_TH, the dark flag r22 is set

## 8.4 Port manipulation

```
ldi  r16, 0b00111111
out  _SFR_IO_ADDR(DDRB), r16      ; PB0-PB5 outputs (traffic LEDs)
ldi  r16, 0b11111000
out   SFR_IO_ADDR(DDRD), r16      ; PD3-PD7 outputs, PD2 input
ldi  r16, 0b00000100
out  _SFR_IO_ADDR(PORTD), r16     ; pull-up on PD2
```

• An entire phase for traffic is configured by setting an output on PORTB through a sequence found in the table above, causing all six lines to be updated simultaneously.

• The countdown updates PORTD with values 0b11111100, 0b01111100, 0b00111100, 0b00011100, and 0b00001100 sequentially, once per second, turning off one LED each time while leaving the pull-up of PD2 intact. The process ends with 0b00000100.

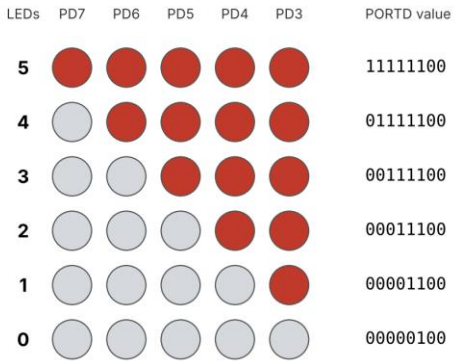



*Figure 11: Countdown LED patterns written to PORTD (red = lit)*

## 8.5 Night mode and hysteresis

Night mode flashes both yellow LEDs for 0.5 s on and 0.5 s off (1 Hz). Every 0.5 s the LDR is read. A value at or above BRIGHT_TH increments r24, any lower value resets it to 0, and when r24 reaches EXIT_COUNT (10, equal to 5 s) the system shows 1 s of all-red and returns to Normal mode at phase 0. Two thresholds (DARK_TH below BRIGHT_TH) add a band that prevents flicker around a single level (Figure 12).

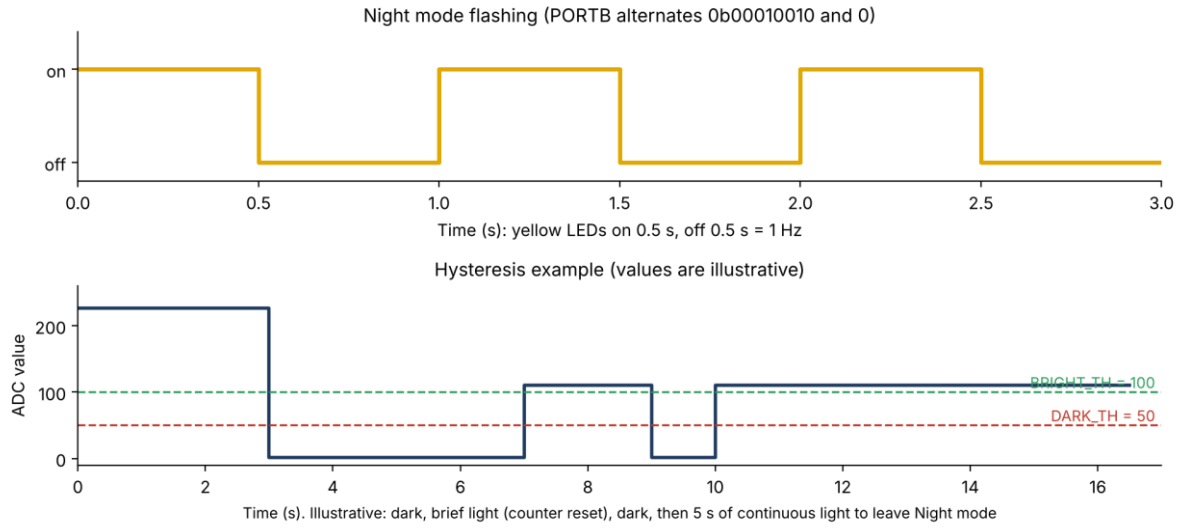



*Figure 12: Night mode flashing and the hysteresis behaviour (illustrative)*

# 9. Testing and Results

Testing has three parts: a measured LDR calibration, a design verification by calculation, and functional tests that follow the demonstration steps in the assignment.

## 9.1 LDR calibration (measured)

The voltage at A0 was measured with a multimeter (DC volts) and converted with ADC = (V / 5) × 255.

| Condition | Voltage at A0 | ADC value | Notes |
|---|---|---|---|
| Room light, LDR uncovered | 4.45 V | ≈ 227 | Measured |
| LDR covered by hand | 0.01 V | ≈ 1 | Measured |
| DARK_TH chosen | ≈ 0.98 V | 50 | About 25 % of the span above the dark reading |
| BRIGHT_TH chosen | ≈ 1.96 V | 100 | About 45 % of the span, above DARK_TH, below the light reading |

The span between the dark and light readings is about 226 counts, so the chosen thresholds leave a large margin on both sides.

## 9.2 Design verification (calculated)

These values follow from the timer configuration and the phase table. They are the behaviour the firmware is designed to produce.

| Item | Calculation | Result |
|---|---|---|
| Timer1 tick | 16 MHz / 256 = 62,500 Hz; OCR1A + 1 = 31,250 | 0.500 s |
| NS or EW green | 10 ticks x 0.5 s | 5 s |
| NS or EW yellow | 4 ticks x 0.5 s | 2 s |
| All-red clearance | 2 ticks x 0.5 s | 1 s |
| Full cycle | 2 x (5 + 2 + 1) | 16 s |
| Pedestrian countdown | 5 steps x 1 s | 5 s |
| Night flash period | 0.5 s on + 0.5 s off | 1 s (1 Hz) |
| Night mode exit | 10 samples x 0.5 s, then 1 s all-red | 5 s + 1 s |
| ADC conversion time | 13 clocks x (1 / 125 kHz) | 104 µs |
| LED current | (5 − 2) V / 330 Ω | ≈ 9 mA |

## 9.3 Functional tests

Each test follows the demonstration steps in the assignment. The result column is completed during the hardware demonstration.

**Mode 1: Normal traffic cycle**

| Test | Expected | Result |
|---|---|---|
| North-South green duration | 5 s | pass |
| North-South yellow duration | 2 s | pass |
| All-red clearance after NS | 1 s | pass |
| East-West green duration | 5 s | pass |
| East-West yellow duration | 2 s | pass |
| All-red clearance after EW | 1 s | pass |
| Phase order, none skipped or repeated | NS green, NS yellow, all-red, EW green, EW yellow, all-red | pass |
| Full cycle time | 16 s | pass |

**Mode 2: Pedestrian crossing**

| Test | Expected | Result |
|---|---|---|
| Press during green | Lights do not change at once | pass |
| Press during yellow | Lights do not change at once | pass |
| Crossing starts only after the all-red interval | Yes | pass |
| All four directions red during crossing | Yes | pass |
| Countdown display | 5 to 0, one step per second | pass |
| Crossing duration | 5 s | pass |
| Several quick presses | Only one crossing | pass |
| Press during the crossing | No second crossing | pass |
| Cycle after the crossing | Resumes at the next phase | pass |

**Mode 3: Night mode**

| Test | Expected | Result |
|---|---|---|
| Cover the LDR | Night mode starts, all four yellows flash | pass |
| Flash rate | 1 Hz (0.5 s on, 0.5 s off) | pass |
| All other LEDs in Night mode | Off | pass |

| Test | Expected | Result |
|---|---|---|
| Uncover the LDR | Normal mode does not return at once | pass |
| Time to return to Normal mode | About 5 s of continuous light, then 1 s all-red | pass |
| Uncover for less than 5 s, then cover again | Night mode continues, counter resets | pass |
| Press the button in Night mode | Ignored | pass |

## 10. Photographs

Insert photographs of the completed junction model from several angles, with a one-line caption under each.

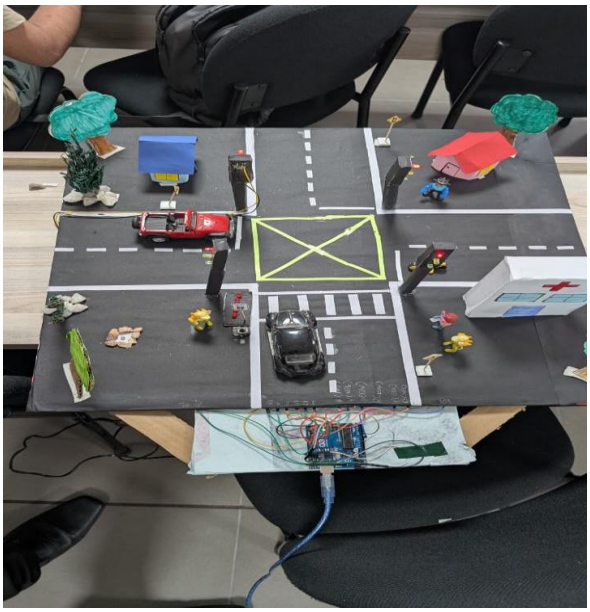


*Figure 13:Top view of the whole board*

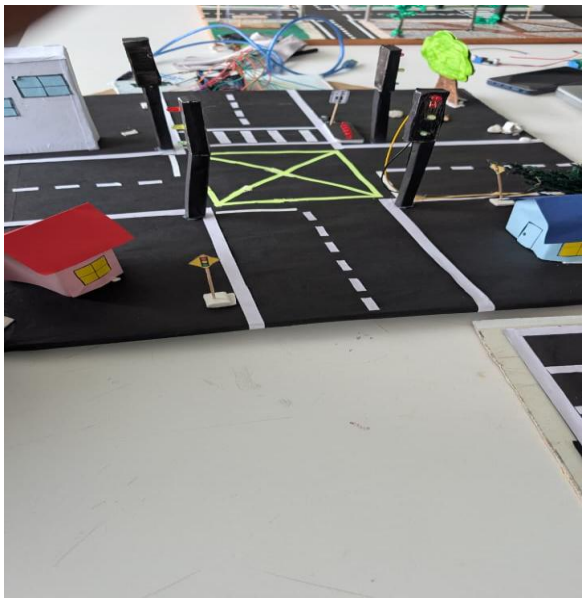


*Figure 14: Side view*

## 11. Discussion

The design meets the three required modes with one source file. The points below record the decisions made and what could be improved.

**Design decisions**

• **The crossing is performed on deferred basis**. ISR just sets a flag. Crossing is done after the all-red cycle, thus there are no sudden changes and overlapping greens are avoided.

• **Only one crossing per request.** The flag is reset as soon as crossing is started, INT0 is disabled in the process of crossing and then the interrupt flag is reset, so subsequent presses do not add another crossing into the queue.

• **Button in Night mode.** Request is ignored: INT0 is disabled and the flag is reset. Pedestrian has to press once again after Normal mode is restored.

• **Two light threshold values plus 5 s timeout.** A short change in light intensity will not switch the mode.

• **Exiting Night mode.** Junction goes through all-red state for 1 s and resets to Phase 0 (NS green) to avoid any unsafe initial state.Limitations

- **Shared pins.** North and South pins cannot be controlled independently; neither can East and West pins.
- **Busy-wait timing.** The processor sits waiting in a loop for the Timer1 flag. A Timer1 Compare Match interrupt will enable the program to sleep.
- **Timing accuracy.** As the timer is reset once per 0.5 seconds following an ADC reading of around 0.1 ms, the time interval is slightly larger than 0.5 seconds (by 0.02%), which is negligible compared to stopwatch accuracy.
- **Light thresholds.** These are dependent on the LDR, the 10 kΩ resistor, and the ambient lighting, and therefore need to be calibrated again if any of these change
- **Countdown patterns.** The five countdown patterns are written out one by one instead of being generated by a loop.

**Possible improvements**

- Separate pins for each approach, using a shift register or an I/O expander.
- A pedestrian indicator LED, and a queued request in Night mode.
- A Timer1 compare-match interrupt for the tick, so the main loop can sleep between events.

# 12. Individual Contribution Statements

Each member is assigned specific modules. Replace the bracketed fields and edit the split to match the work actually done.

| Member | Name / Student ID | Modules and tasks |
|---|---|---|
| Member 1 | Fernando M V M | pedestrian crossing, INT0 handling, countdown display, circuit design and model build |
| Member 2 | M T M Dushan | traffic light cycling, phase table, Night mode, ADC reading, Timer1 configuration |

# References


- Microchip Technology, ATmega48A/PA/88A/PA/168A/PA/328/P Data Sheet (DS40002061).
- Microchip Technology, AVR Instruction Set Manual (DS40002198).

- GNU Binutils, Using as: AVR dependent features.
- Module notes and assignment brief, IE3064 Embedded Systems Engineering, SLIIT.